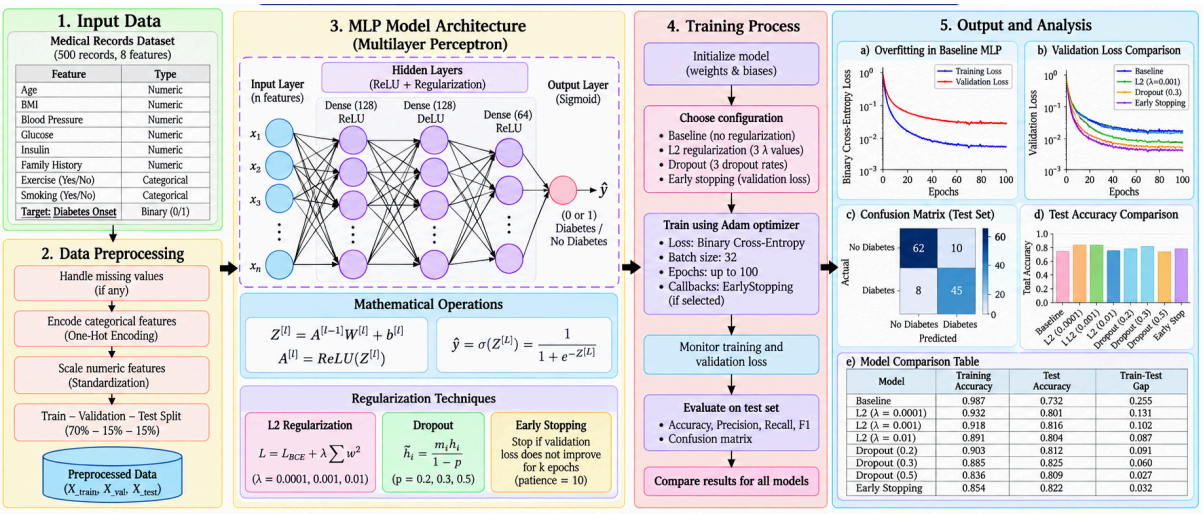
Experiment 10: L2, Dropout and Early Stopping Comparison

1. Aim

To investigate overfitting in a large Multilayer Perceptron (MLP) trained on a diabetes prediction dataset. To compare L2 regularization with different λ values, dropout with different rates, and early stopping based on validation loss. To evaluate each model using test accuracy and the training–test accuracy gap with TensorFlow, Keras, NumPy, and Matplotlib.



2. Mathematical Formulas

1. Binary Cross-Entropy Loss

$$\boxed{L_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(p_i) + (1-y_i)\log(1-p_i)]}$$

2. L2 Regularization

$$\boxed{L_{\text{total}} = L_{\text{BCE}} + \lambda \sum_{j-1}^{m} w_j^2}$$

where λ controls the regularization strength and w_j are the model weights.

3. Dropout

$$\boxed{\tilde{h}_i = \frac{m_i h_i}{1-r}}$$

where m_i is a Bernoulli random variable and r is the dropout rate during training. Dropout randomly disables a fraction of neurons during training to reduce over-reliance on particular neurons.

4. Early Stopping

$$\boxed{t^* = \arg\min_t L_{\text{validation}}(t)}$$

Training stops when validation loss fails to improve for a specified number of epochs (patience).

5. Accuracy

$$\boxed{\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}}$$

6. Training–Test Accuracy Gap

$$\boxed{\text{Gap} = \text{Training Accuracy} - \text{Test Accuracy}}$$

CODE

```
# Experiment 10: L2, Dropout and Early Stopping Comparison

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

np.random.seed(42)
tf.random.set_seed(42)

# ============================================
# 1. GENERATE 500 SYNTHETIC PATIENT RECORDS
# ============================================

n = 500
X = np.random.normal(size=(n, 8))

# Illustrative binary diabetes label
logit = (
    0.9 * X[:, 0]
    + 0.7 * X[:, 1]
    + 0.5 * X[:, 2]
    - 0.4 * X[:, 3]
    + 0.5 * X[:, 0] * X[:, 1]
    + np.random.normal(0, 0.7, n)
)

prob = 1 / (1 + np.exp(-logit))
y = (np.random.random(n) < prob).astype(int)

print("Dataset shape:", X.shape)
print("Patient records:", n)
print("Positive cases:", np.sum(y == 1))
print("Negative cases:", np.sum(y == 0))

# ============================================
# 2. SPLIT AND STANDARDIZE THE DATA
# ============================================

X_train_full, X_test, y_train_full, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

X_train, X_val, y_train, y_val = train_test_split(
    X_train_full, y_train_full,
    test_size=0.20, random_state=42, stratify=y_train_full
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)
```

```
print("\nTraining records:", len(X_train))
print("Validation records:", len(X_val))
print("Test records:", len(X_test))

# ============================================
# 3. BUILD THE MLP
# ============================================

def build_model(l2_value=0.0, dropout_rate=0.0):
    regularizer = (
        tf.keras.regularizers.l2(l2_value)
        if l2_value > 0 else None
    )

    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(X_train.shape[1],)),

        tf.keras.layers.Dense(
            128, activation="relu",
            kernel_regularizer=regularizer
        ),
        tf.keras.layers.Dropout(dropout_rate),

        tf.keras.layers.Dense(
            128, activation="relu",
            kernel_regularizer=regularizer
        ),
        tf.keras.layers.Dropout(dropout_rate),

        tf.keras.layers.Dense(
            64, activation="relu",
            kernel_regularizer=regularizer
        ),
        tf.keras.layers.Dense(1, activation="sigmoid")
    ])

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    return model

# ============================================
# 4. TRAIN AND EVALUATE EACH MODEL
# ============================================

results = []
histories = {}
trained_models = {}

def train_evaluate(name, l2_value=0.0,
                   dropout_rate=0.0, early_stop=False):

    tf.keras.backend.clear_session()
    tf.random.set_seed(42)
```

```
    model = build_model(l2_value, dropout_rate)

    callbacks = []
    if early_stop:
        callbacks.append(
            tf.keras.callbacks.EarlyStopping(
                monitor="val_loss",
                patience=10,
                restore_best_weights=True
            )
        )

    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=100,
        batch_size=32,
        callbacks=callbacks,
        verbose=0
    )

    # Evaluate accuracy on training and test datasets
    train_prob = model.predict(X_train, verbose=0).ravel()
    test_prob = model.predict(X_test, verbose=0).ravel()

    train_pred = (train_prob >= 0.5).astype(int)
    test_pred = (test_prob >= 0.5).astype(int)

    train_acc = accuracy_score(y_train, train_pred)
    test_acc = accuracy_score(y_test, test_pred)

    results.append({
        "Model": name,
        "Epochs": len(history.history["loss"]),
        "Training Accuracy": train_acc,
        "Test Accuracy": test_acc,
        "Train-Test Gap": train_acc - test_acc
    })

    histories[name] = history.history
    trained_models[name] = model

    print(
        f"{name}: Test Accuracy = {test_acc:.4f}, "
        f"Gap = {train_acc - test_acc:.4f}"
    )

# Baseline large MLP
train_evaluate("Baseline")

# L2 regularization: three lambda values
for lam in [0.0001, 0.001, 0.01]:
    train_evaluate(f"L2 lambda={lam}", l2_value=lam)

# Dropout: three rates
for rate in [0.2, 0.3, 0.5]:
    train_evaluate(f"Dropout rate={rate}", dropout_rate=rate)
```

```python
# Early stopping
train_evaluate("Early Stopping", early_stop=True)

# ============================================
# 5. DISPLAY COMPARISON TABLE
# ============================================

results_df = pd.DataFrame(results)

print("\nModel Comparison:")
print(results_df.round(4).to_string(index=False))

# ============================================
# 6. PLOT BASELINE OVERFITTING
# ============================================

baseline = histories["Baseline"]

plt.figure(figsize=(8, 5))
plt.plot(baseline["loss"], label="Training Loss")
plt.plot(baseline["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy Loss")
plt.title("Baseline MLP: Overfitting Analysis")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# ============================================
# 7. COMPARE VALIDATION LOSS CURVES
# ============================================

plt.figure(figsize=(11, 6))

for name, history in histories.items():
    plt.plot(history["val_loss"], label=name)

plt.xlabel("Epoch")
plt.ylabel("Validation Loss")
plt.title("Validation Loss: Regularization Comparison")
plt.legend(fontsize=8)
plt.grid(alpha=0.3)
plt.show()

# ============================================
# 8. COMPARE TEST ACCURACY
# ============================================

plt.figure(figsize=(11, 5))
plt.bar(
    results_df["Model"],
    results_df["Test Accuracy"]
)

plt.xticks(rotation=45, ha="right")
plt.ylabel("Test Accuracy")
plt.title("Test Accuracy of All Models")
```

```
plt.tight_layout()
plt.show()

# ============================================
# 9. COMPARE TRAIN-TEST GAPS
# ============================================

plt.figure(figsize=(11, 5))
plt.bar(
    results_df["Model"],
    results_df["Train-Test Gap"]
)

plt.axhline(0, color="black", linewidth=1)
plt.xticks(rotation=45, ha="right")
plt.ylabel("Training Accuracy - Test Accuracy")
plt.title("Generalization Gap Comparison")
plt.tight_layout()
plt.show()

# Best model by test accuracy (for this experiment)
best = results_df.loc[results_df["Test Accuracy"].idxmax()]

print("\nBest test accuracy in this run:")
print(best.to_string())
```

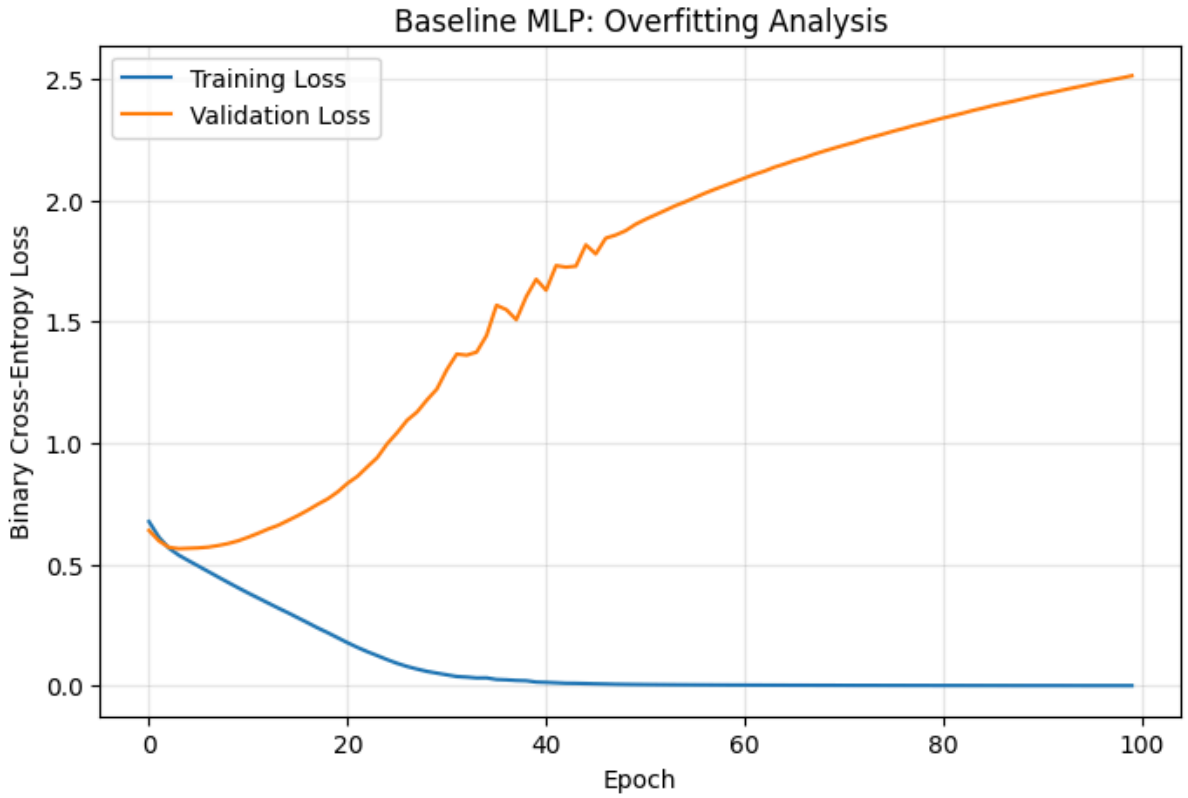
OUTPUT

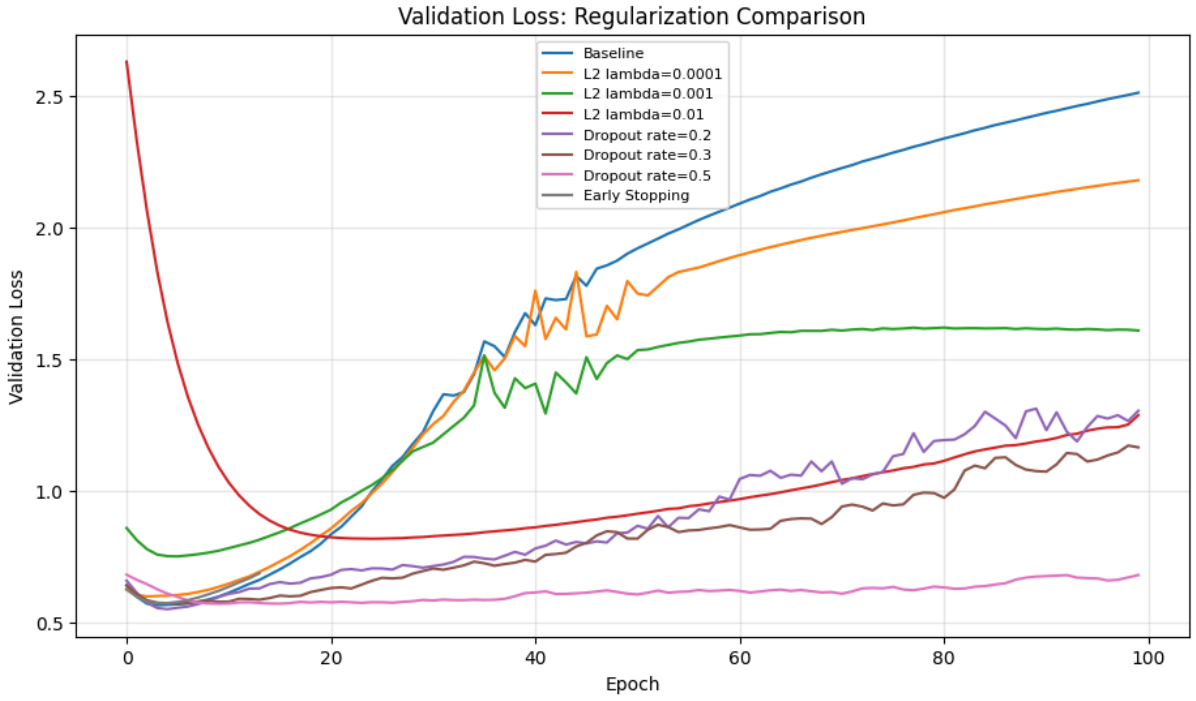
Dataset shape: (500, 8)
Patient records: 500
Positive cases: 244
Negative cases: 256

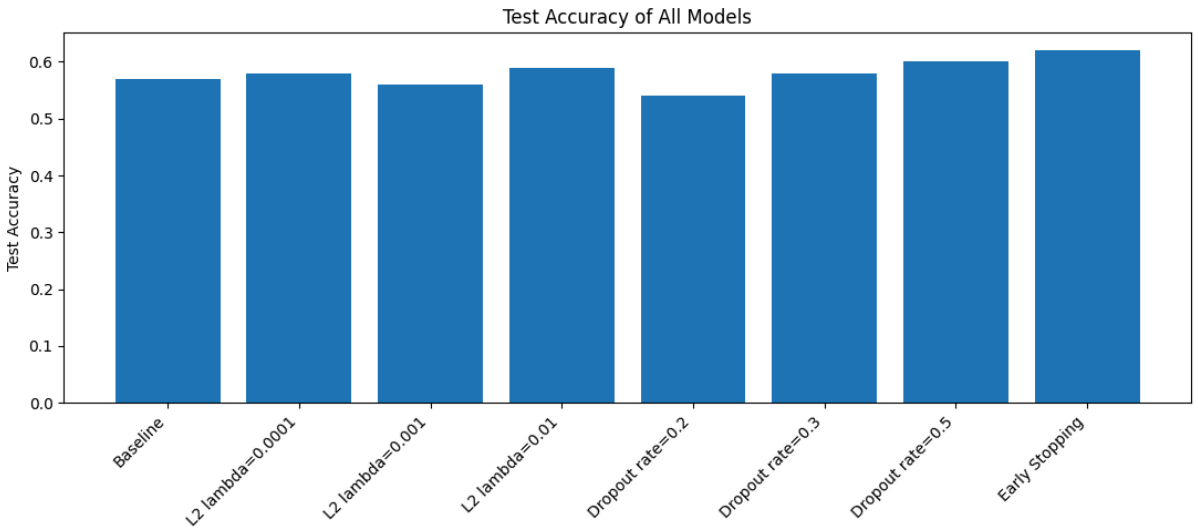
Training records: 320
Validation records: 80
Test records: 100
Baseline: Test Accuracy = 0.5700, Gap = 0.4300
L2 lambda=0.0001: Test Accuracy = 0.5800, Gap = 0.4200
L2 lambda=0.001: Test Accuracy = 0.5600, Gap = 0.4400
L2 lambda=0.01: Test Accuracy = 0.5900, Gap = 0.3663
Dropout rate=0.2: Test Accuracy = 0.5400, Gap = 0.4600
Dropout rate=0.3: Test Accuracy = 0.5800, Gap = 0.4138
Dropout rate=0.5: Test Accuracy = 0.6000, Gap = 0.2594
Early Stopping: Test Accuracy = 0.6200, Gap = 0.1144

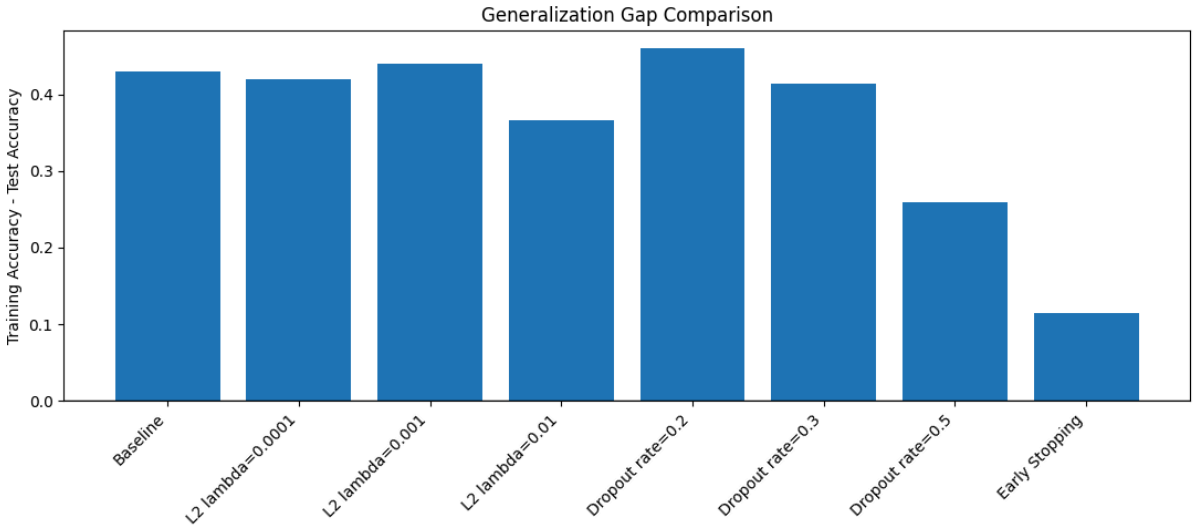
Model Comparison:

Model	Epochs	Training Accuracy	Test Accuracy	Train-Test Gap
Baseline	100	1.0000	0.57	0.4300
L2 lambda=0.0001	100	1.0000	0.58	0.4200
L2 lambda=0.001	100	1.0000	0.56	0.4400
L2 lambda=0.01	100	0.9562	0.59	0.3663
Dropout rate=0.2	100	1.0000	0.54	0.4600
Dropout rate=0.3	100	0.9938	0.58	0.4138
Dropout rate=0.5	100	0.8594	0.60	0.2594
Early Stopping	14	0.7344	0.62	0.1144


Baseline MLP: Overfitting Analysis
Training Loss
Validation Loss
Binary Cross-Entropy Loss
Epoch
0.0
0.5
1.0
1.5
2.0
2.5
0
20
40
60
80
100


Validation Loss: Regularization Comparison
Baseline
L2 lambda=0.0001
L2 lambda=0.001
L2 lambda=0.01
Dropout rate=0.2
Dropout rate=0.3
Dropout rate=0.5
Early Stopping
Validation Loss
Epoch
0.5
1.0
1.5
2.0
2.5
0
20
40
60
80
100





Best test accuracy in this run:

Model	Early Stopping
Epochs	14
Training Accuracy	0.734375
Test Accuracy	0.62
Train-Test Gap	0.114375

5. RESULT

A large baseline MLP and models using L2 regularization, dropout, and early stopping were implemented using TensorFlow and Keras. Training and validation loss curves were plotted to investigate overfitting. The models were compared using test accuracy and the train–test accuracy gap. The experiment demonstrates how regularization techniques and early stopping can help improve generalization. The most suitable model is selected based on its test performance and generalization gap.